

선박 수익성 분석

군집화 방법론을 중심으로

목차

01 개요

- 분석 목적

02 데이터 구조

- 데이터 구조
- 데이터 전처리
- EDA

03 분석 방법론

- 군집분석 개요
- 클러스터링 기법

04 실증 분석

- 군집분석 결과
- 클러스터별 특성

05 결론

- 클러스터 간 비교
- PCA
- 실무적 활용 방안

분석 목적

선박 성능 지표를 활용한 선박 그룹화

- 선박의 다양한 성능 지표 기반 군집화
- 객관적 데이터에 기반한 선박 그룹 분류 방법론 제시

군집별 수익성 비교 분석

- 성능 그룹별 운항 수익성 및 비용 구조 차이 분석
- 선박 포트폴리오 최적화를 위한 정량적 근거 제공



의사결정 지원

- 선주 및 운영사의 선박 투자 의사결정에 객관적 기준 제공
- 성능 지표와 수익성 간의 상관관계 분석을 통한 전략적 판단 지원
- 선박 운영 효율화를 위한 데이터 기반 의사결정 체계 구축

데이터 구조

선박 성능 데이터셋은 기니만에서 다양한 선박 유형의 주요 운영 지표와 속성을 나타내도록 설계된 데이터 셋입니다. 2,736개의 행과 24개의 열로 구성되어 있으며, 수치형과 범주형으로 분류됩니다. 주요 특징은 다음과 같습니다.

수치형 데이터

- Speed_Over_Ground_knots: 선박의 수면 위 평균 속도(노트)
- Engine_Power_kW: 엔진 출력(킬로와트)
- Distance_Traveled_nm: 선박이 이동한 총 거리(해리)
- Operational_Cost_USD: 항해당 총 운영 비용(USD)
- Revenue_per_Voyage_USD: 항해당 발생한 수익(USD)
- Efficiency_nm_per_kWh: 킬로와트시당 해리로 계산된 에너지 효율

범주형 데이터

- Ship_Type: 선박 유형(유조선, 컨테이너선, 어선, 벌크선)
- Route_Type: 운송 경로 유형(단거리, 장거리, 대양 횡단)
- Engine_Type: 엔진 유형(디젤, 중유)
- Maintenance_Status: 선박의 유지 관리 상태(보통, 심각, 양호)
- 날씨 상황: 항해 중 나타나는 날씨 상황(고요함, 보통, 험함)

데이터 전처리

결측치 및 이상치 처리

- 이상치는 존재하지 않음
- 범주형 변수에서만 결측치 존재

```
# === 컬럼별 이상치 개수 (IQR 기준, 숫자형만) ===
num_df = df.select_dtypes(include=['number'])
Q1 = num_df.quantile(0.25)
Q3 = num_df.quantile(0.75)
IQR = Q3 - Q1
outlier_count = (((num_df < (Q1 - 1.5*IQR)) | (num_df > (Q3 + 1.5*IQR)))).sum()
outlier_count
```

Speed_Over_Ground_knots	0
Engine_Power_kW	0
Distance_Traveled_nm	0
Draft_meters	0
Cargo_Weight_tons	0
Operational_Cost_USD	0
Revenue_per_Voyage_USD	0
Turnaround_Time_hours	0
Efficiency_nm_per_kWh	0
Seasonal_Impact_Score	0
Weekly_Voyage_Count	0
Average_Load_Percentage	0

dtype: int64

```
# 결측치 개수
df.isna().sum()
```

Date	0
Ship_Type	136
Route_Type	136
Engine_Type	136
Maintenance_Status	136
Speed_Over_Ground_knots	0
Engine_Power_kW	0
Distance_Traveled_nm	0
Draft_meters	0
Weather_Condition	136
Cargo_Weight_tons	0
Operational_Cost_USD	0
Revenue_per_Voyage_USD	0
Turnaround_Time_hours	0
Efficiency_nm_per_kWh	0
Seasonal_Impact_Score	0
Weekly_Voyage_Count	0
Average_Load_Percentage	0

dtype: int64

데이터 전처리

결측치 및 이상치 처리

- 범주형 결측치는 KNN기법을 사용해 예측하여 채움

```
# 0) 작업용 복사본
df_knn = df.copy()

# 1) 컬럼 구분
num_cols = df_knn.select_dtypes(include=['number']).columns.tolist()
cat_cols = df_knn.select_dtypes(include=['object']).columns.tolist()

# 2) 결측이 있는 범주형 컬럼만 타깃으로 선정
cat_cols_with_na = [c for c in cat_cols if df_knn[c].isna().any()]
print("결측 있는 범주형 컬럼:", cat_cols_with_na)

# 3) 각 범주형 컬럼을 '하나씩' 타깃으로 삼아 KNN 분류로 채우기
filled_summary = []

# 3) 각 범주형 컬럼을 '하나씩' 타깃으로 삼아 KNN 분류로 채우기
filled_summary = []

for target_col in cat_cols_with_na:
    # (a) 학습/예측 마스크
    mask_train = df_knn[target_col].notna()
    mask_pred = df_knn[target_col].isna()

    if mask_pred.sum() == 0:
        continue # 채울 게 없으면 스킵

    # (b) 입력 특성: 타깃 컬럼은 제외하고 나머지를 사용
    # - 숫자형: Robust 스케일링
    # - 범주형: (타깃 제외) 원핫 인코딩
    other_cat = [c for c in cat_cols if c != target_col]
    preproc = ColumnTransformer(
        transformers=[
            ("num", RobustScaler(), num_cols),
            ("cat", OneHotEncoder(handle_unknown="ignore"), other_cat),
        ],
        remainder='drop'
    )

    # (c) 파이프라인: 전처리 + KNN 분류
    # n_neighbors는 5로 시작, 데이터 밀도나 성능 보며 조정 가능
    pipe = Pipeline(steps=[
        ("prep", preproc),
        ("clf", KNeighborsClassifier(n_neighbors=5, weights='distance'))
    ])

    # (d) 학습(결측 아닌 행) / 예측(결측 행)
    X_train = df_knn.loc[mask_train, num_cols + other_cat]
    y_train = df_knn.loc[mask_train, target_col]

    X_pred = df_knn.loc[mask_pred, num_cols + other_cat]
```

```
# 원-핫 인코딩 전에 결측이 남아있으면 인코더가 예러 → 안전하게 임시 대체
# - 다른 범주형의 결측은 "Missing"으로 잠깐 채워 넣은 뒤 인코딩
X_train = X_train.copy()
X_pred = X_pred.copy()
for c in other_cat:
    X_train[c] = X_train[c].fillna("Missing")
    X_pred[c] = X_pred[c].fillna("Missing")
# 숫자형 결측은 그대로 두면 전처리에서 NaN이 남아 예러 → 중앙값으로 '임시' 대체
# (숫자형 결측을 영구적으로 채우는 건 이 단계의 목적이 아니고, KNN이 돌기 위한 임시조치)
for c in num_cols:
    med = df_knn[c].median() if np.isfinite(df_knn[c].median()) else 0
    X_train[c] = X_train[c].fillna(med)
    X_pred[c] = X_pred[c].fillna(med)

pipe.fit(X_train, y_train)
y_hat = pipe.predict(X_pred)

# (e) 채우기
n_fill = mask_pred.sum()
df_knn.loc[mask_pred, target_col] = y_hat

filled_summary.append((target_col, int(n_fill), list(pd.Series(y_hat).value_counts().index)))

# 4) 리포트
if filled_summary:
    rep = pd.DataFrame(filled_summary, columns=["column", "filled_rows", "predicted_classes(top order)"])
    print("KNN으로 채운 결과 요약:")
    display(rep)
else:
    print("채울 범주형 결측치가 없습니다.")

# ▶ 결과 데이터프레임: df_knn (이후 단계에서는 df_knn을 사용하세요)
```

결측 있는 범주형 컬럼: ['Ship_Type', 'Route_Type', 'Engine_Type', 'Maintenance_Status', 'Weather_Condition']
KNN으로 채운 결과 요약:

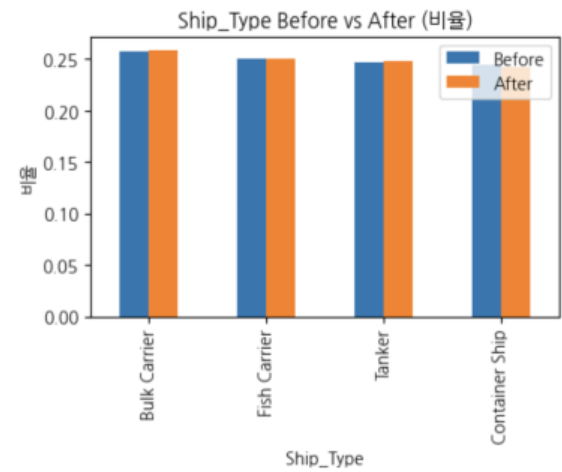
	column	filled_rows	predicted_classes(top order)
0	Ship_Type	136	[Bulk Carrier, Tanker, Fish Carrier, Container...]
1	Route_Type	136	[Short-haul, Transoceanic, Long-haul, Coastal]
2	Engine_Type	136	[Steam Turbine, Diesel, Heavy Fuel Oil (HFO)]
3	Maintenance_Status	136	[Fair, Good, Critical]
4	Weather_Condition	136	[Rough, Moderate, Calm]

데이터 전처리

결측치 및 이상치 처리 - KNN기법을 결과

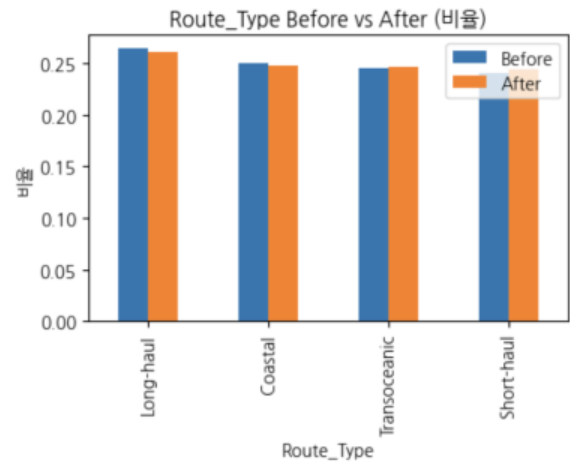
Ship_Type 분포 변화 (KNN 대체 전 vs 후)

	Before	After	Change(%)
Ship_Type			
Bulk Carrier	0.257308	0.259137	0.18
Fish Carrier	0.251154	0.250731	-0.04
Tanker	0.247308	0.248173	0.09
Container Ship	0.244231	0.241959	-0.23



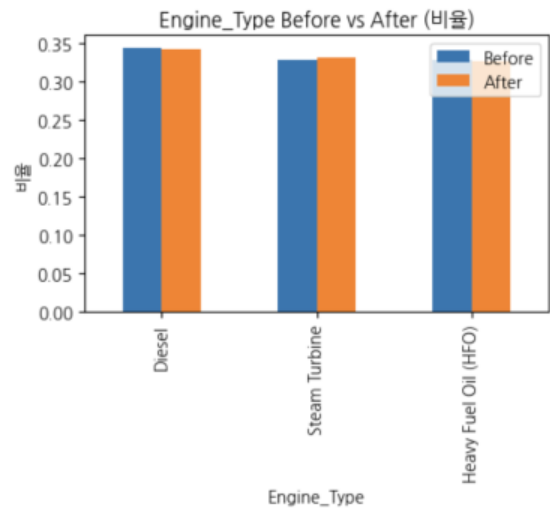
Route_Type 분포 변화 (KNN 대체 전 vs 후)

	Before	After	Change(%)
Route_Type			
Long-haul	0.263846	0.261330	-0.25
Coastal	0.250000	0.247807	-0.22
Transoceanic	0.245385	0.246345	0.10
Short-haul	0.240769	0.244518	0.37



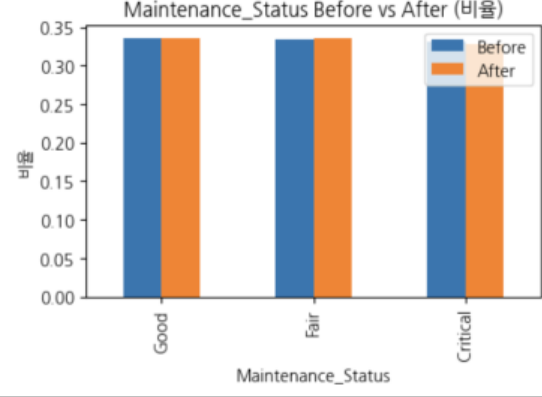
Engine_Type 분포 변화 (KNN 대체 전 vs 후)

	Before	After	Change(%)
Engine_Type			
Diesel	0.343077	0.342105	-0.10
Steam Turbine	0.328846	0.330775	0.19
Heavy Fuel Oil (HFO)	0.328077	0.327120	-0.10



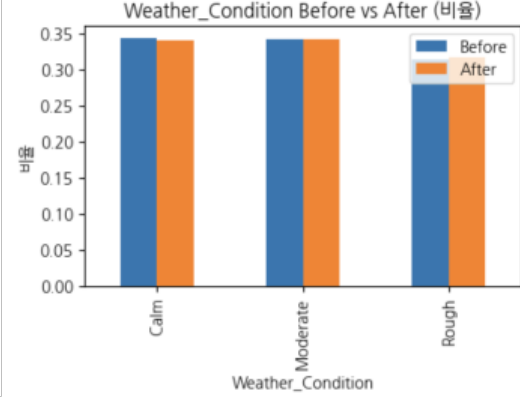
Maintenance_Status 분포 변화 (KNN 대체 전 vs 후)

	Before	After	Change(%)
Maintenance_Status			
Good	0.335769	0.335892	0.01
Fair	0.333462	0.336257	0.28
Critical	0.330769	0.327851	-0.29



Weather_Condition 분포 변화 (KNN 대체 전 vs 후)

	Before	After	Change(%)
Weather_Condition			
Calm	0.343462	0.341009	-0.25
Moderate	0.342692	0.342471	-0.02
Rough	0.313846	0.316520	0.27

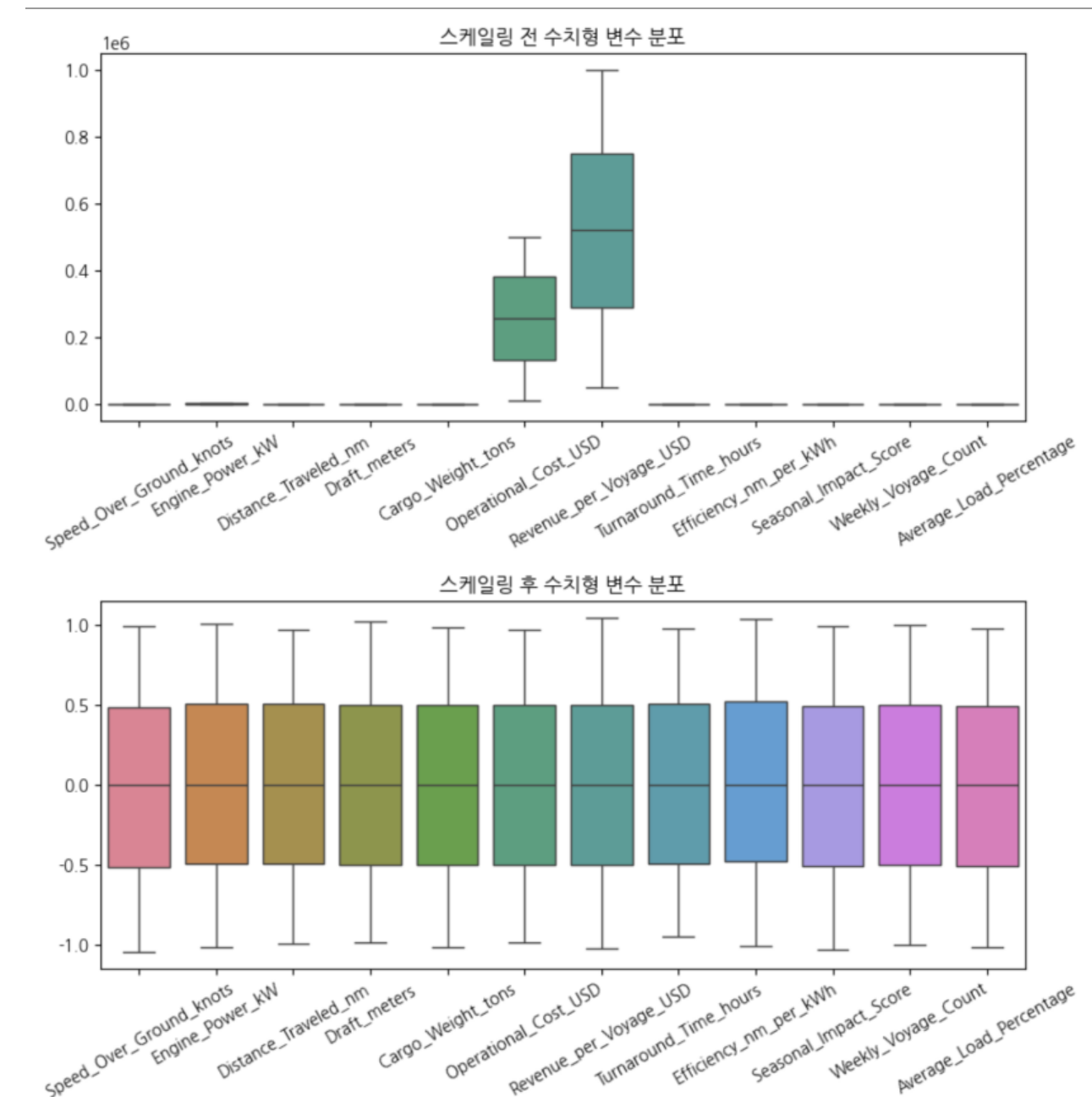


데이터 전처리

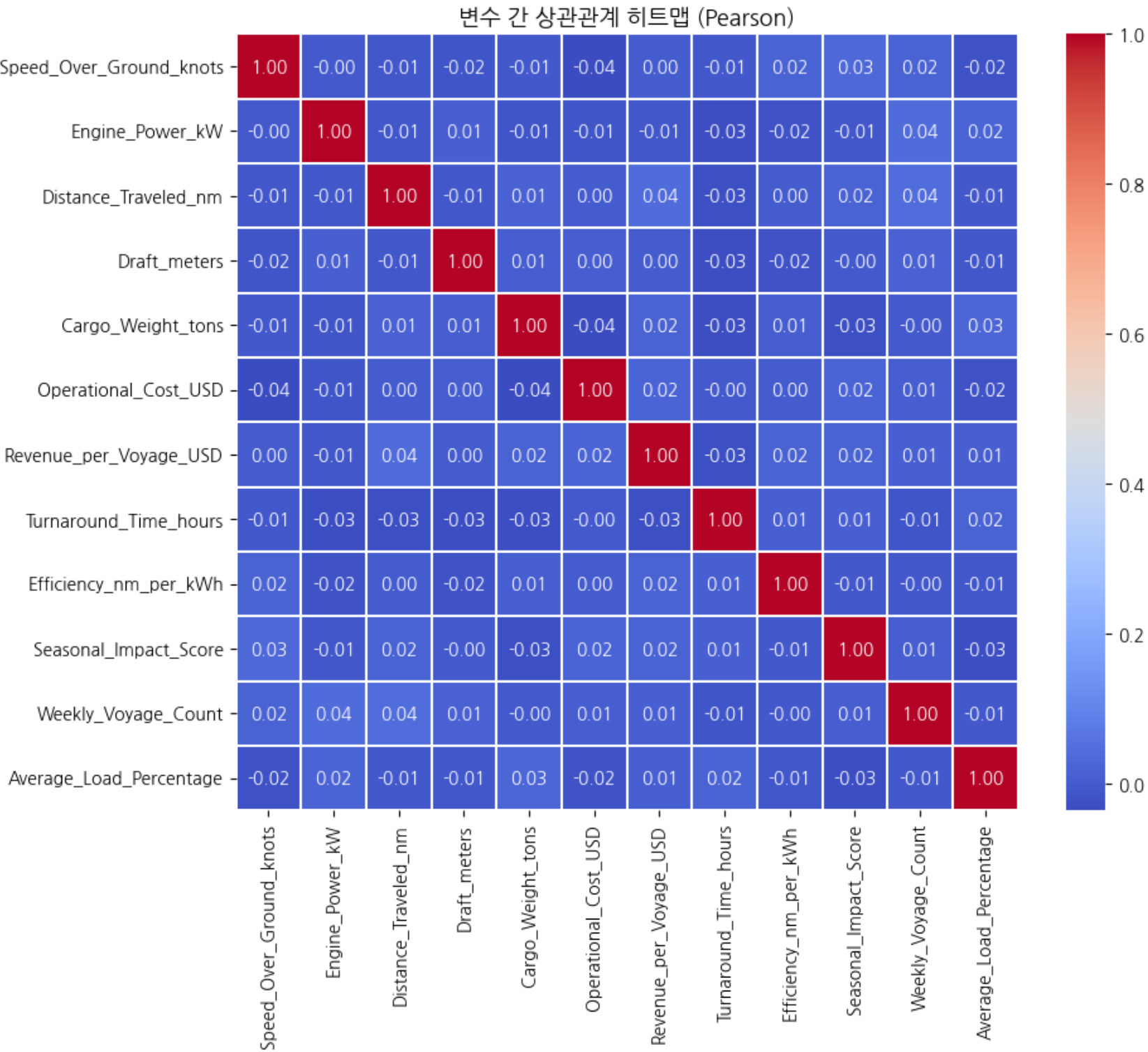
원-핫 인코딩 및 데이터 스케일링

- 범주형 변수는 원-핫 인코딩(Date 칼럼은 인코딩 하지 않음)
- 수치형 변수는 스케일링(RobustScaler 적용)

	변수명	고유값개수	예상_원핫컬럼수
0	Date	57	57
1	Ship_Type	4	4
2	Route_Type	4	4
3	Engine_Type	3	3
4	Maintenance_Status	3	3
5	Weather_Condition	3	3



주요 변수간 상관관계
- 피어슨 상관계수 분석 진행



분석 방법론

군집분석을 선택한 이유

- 회귀분석을 통한 수익 예측 시도

Saving 배.csv to 배.csv

📁 데이터 로드 완료: (2736, 33)

Fitting 5 folds for each of 27 candidates, totalling 135 fits

✅ [Gradient Boosting Regressor 결과]

Best Params: {'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 100}

R²: -0.007768090643133663

MAE: 250640.72642519194

Saving 배.csv to 배 (1).csv

📁 데이터 로드 완료: (2736, 33)

Fitting 5 folds for each of 27 candidates, totalling 135 fits

✅ [Random Forest Regressor 결과]

Best Params: {'max_depth': 5, 'min_samples_split': 2, 'n_estimators': 200}

R²: -0.017381677561083553

MAE: 251533.37170662257

Saving 배.csv to 배 (2).csv

📁 데이터 로드 완료: (2736, 33)

Fitting 5 folds for each of 108 candidates, totalling 540 fits

✅ [XGBoost Regressor 결과]

Best Params: {'colsample_bytree': 0.8, 'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 100, 'subsample': 0.8}

R²: -0.009487394988644038

MAE: 250510.63910926966

군집분석 개요

- 유사한 특성을 가진 데이터를 그룹화하는 비지도 학습 방법
- 선박 성능 지표 기반으로 유사한 성능 특성을 가진 선박들을 분류
 - 내부 유사성 최대화, 외부 차별성 최대화 원칙 적용
 - 선박 간 성능 차이를 객관적으로 분석 가능

K-Means 클러스터링

- 가장 널리 사용되는 분할 군집화 알고리즘
- 사전에 지정된 k개의 중심점을 기준으로 군집 형성
- 각 데이터 포인트를 가장 가까운 중심점에 할당
- 선박 성능 지표의 명확한 구분이 필요할 때 적합

MeanShift 클러스터링

- 밀도를 따라 중심을 이동시켜 군집을 찾는 알고리즘
- 알고리즘이 데이터 분포를 보고 군집 수를 찾아줌
- 결과적으로 확률 밀도가 가장 높은 지점에 수렴
- 밀도 기반이라 노이즈·이상치에 비교적 강함

GMM 클러스터링

- 데이터가 클러스터에 속할 확률을 찾는 소프트 알고리즘
- 경계 부근 점들의 애매함을 자연스럽게 표현
 - 다양한 분산 구조를 모델링할 수 있음
- 클러스터 개수를 사전에 정해야 하며, K 선택에 민감함

계층적 클러스터링

- 클러스터 개수를 사전에 정할 필요가 없음
 - 군집 개수를 유연하게 정할 수 있음
- 데이터 구조 파악·설명·이상치 탐지에 유용
- 초기 단계의 잘못된 결합이 이후 전체 구조에 영향을 줌

분석 방법론

클러스터링 기법 - Kmeans 및 GMM

```
# 1) 데이터 불러오기
df = pd.read_csv("배.csv")

# 2) 타겟(y) 제거
target_col = "Net_Profit_USD"
X = df.drop(columns=[target_col])

# 3) 스케일링
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4) k = 2 ~ 10 엘보우 + 실루엣 계산
inertias = []
silhouette_scores = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(
        n_clusters=k,
        init='k-means++',
        n_init=10,
        max_iter=300,
        random_state=42
    )
    labels = kmeans.fit_predict(X_scaled)

    inertias.append(kmeans.inertia_)
    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)

# =====
# 1. 데이터 불러오기 & X / y 분리
# =====

df = pd.read_csv("배.csv")

print("데이터 크기:", df.shape)
print("컬럼 목록:", df.columns.tolist())

# --- y 분리: 군집화엔 쓰지 않고 나중에 결과 비교용으로만 사용 ---
target_col = "Net_Profit_USD"

if target_col in df.columns:
    X = df.drop(columns=[target_col]) # 설명변수
    y = df[target_col] # 결과변수
    print(f"\n'{target_col}' 컬럼을 y로 분리하고, X에서 제외했습니다.")
else:
    X = df.copy()
    y = None
    print(f"\n경고: '{target_col}' 컬럼이 없어 전체 데이터를 X로 사용합니다.")

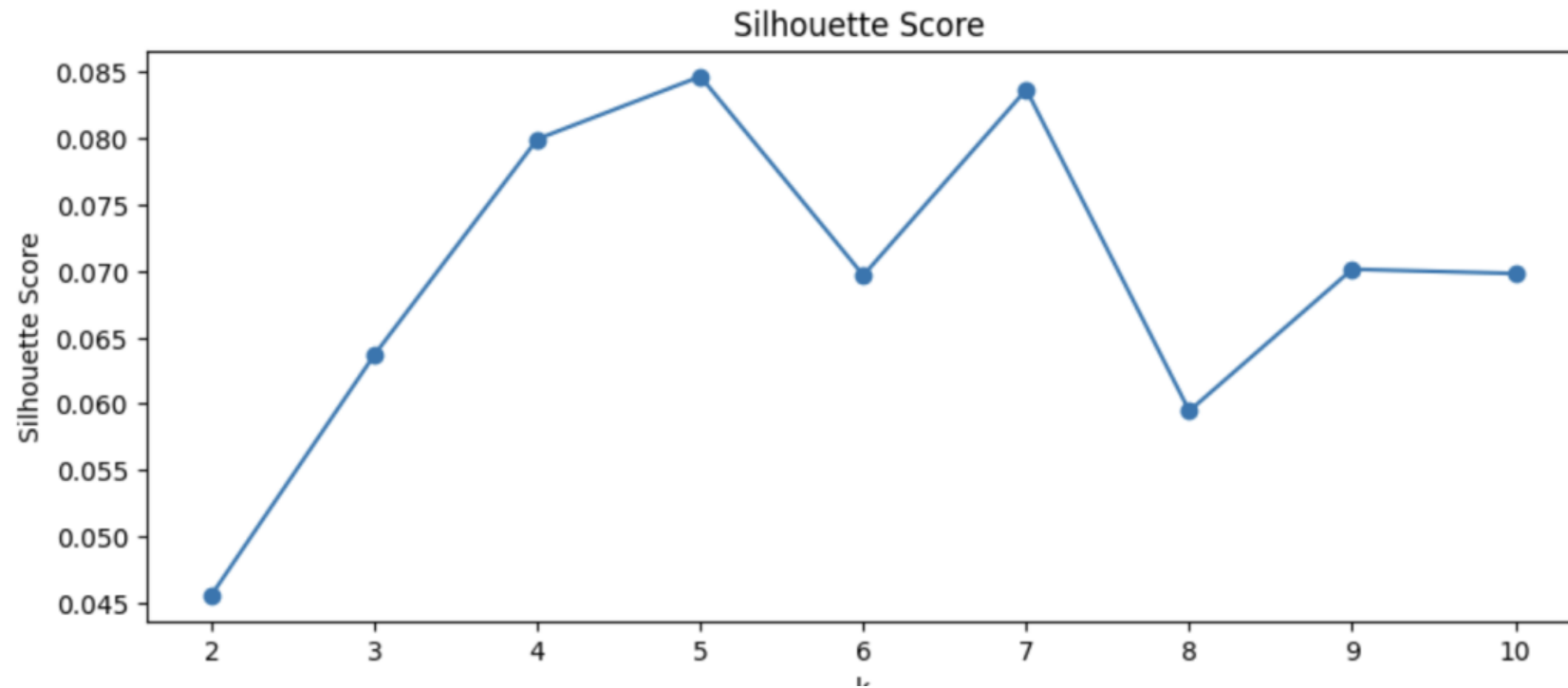
print("설명변수 X 크기:", X.shape)

# =====
# 2. GMM 설정: K, covariance_type 후보 정의
# =====

# 군집 수 후보 K: 필요에 따라 범위 조절 가능
k_range = range(2, 9) # K = 2,3,4,5,6,7,8

# 공분산 구조 후보 (필요하면 줄이거나 늘려도 됨)
covariance_types = ["full", "tied", "diag", "spherical"]

print("\n탐색할 K 값:", list(k_range))
print("탐색할 covariance_type:", covariance_types)
```



```
# =====
# 3. 각 (K, cov_type) 조합마다 GMM 적합 + AIC/BIC + Silhouette 계산
# =====

results = []

X_array = X.values # sklearn은 numpy array를 선호하므로

for cov_type in covariance_types:
    for k in k_range:
        # GMM 모델 정의
        gmm = GaussianMixture(
            n_components=k,
            covariance_type=cov_type,
            n_init=5, # 서로 다른 초기값으로 5번 시도
            reg_covar=1e-6,
            random_state=42
        )

        # 모델 적합
        gmm.fit(X_array)

        # 하드 라벨. (실루엣스코어계산위해서) (가장 확률 높은 군집으로 할당)
        labels = gmm.predict(X_array)

        # AIC, BIC 계산 (작을수록 좋은 모델)
        aic = gmm.aic(X_array)
        bic = gmm.bic(X_array)

        # Silhouette 점수 계산 (클수록 군집 분리도가 좋음)
        # K>=2라서 기본적으로 계산 가능
        sil = silhouette_score(X_array, labels)

        results.append({
            "k": k,
            "covariance_type": cov_type,
            "AIC": aic,
            "BIC": bic,
            "Silhouette": sil
        })

print(f"[cov={cov_type:8s}, k={k}] AIC={aic:.1f}, BIC={bic:.1f}, Silhouette={sil:.4f}")
```

```
=== 전체 결과 (앞부분만) ===
k covariance_type AIC BIC Silhouette
0 2 full -144150.103217 -137520.226413 0.022643
1 3 full -192816.416499 -182868.644166 0.018671
2 4 full -190851.714242 -177586.046380 0.042989
3 5 full -201133.129533 -184549.566143 0.043312
4 6 full -210377.887796 -190476.428878 0.032377

=== BIC 기준 상위 5개 모델 ===
k covariance_type AIC BIC Silhouette
5 7 full -223358.008376 -200138.653930 0.036685
6 8 full -221751.826469 -195214.576495 0.049647
4 6 full -210377.887796 -190476.428878 0.032377
3 5 full -201133.129533 -184549.566143 0.043312
1 3 full -192816.416499 -182868.644166 0.018671

=== AIC 기준 상위 5개 모델 ===
k covariance_type AIC BIC Silhouette
5 7 full -223358.008376 -200138.653930 0.036685
6 8 full -221751.826469 -195214.576495 0.049647
4 6 full -210377.887796 -190476.428878 0.032377
3 5 full -201133.129533 -184549.566143 0.043312
1 3 full -192816.416499 -182868.644166 0.018671

=== Silhouette 기준 상위 5개 모델 ===
k covariance_type AIC BIC Silhouette
9 4 tied -164123.135798 -160225.643547 0.067760
8 3 tied -131170.397007 -127468.075081 0.064258
15 3 diag -8449.536850 -7302.171908 0.064258
22 3 spherical 114205.867840 114803.207320 0.063616
27 8 spherical 111889.161817 113491.924184 0.056176
```

실루엣 계수가
너무 낮음!

분석 방법론

클러스터링 기법

- MeanShift

```
# -----  
# 평균 이동(Mean Shift) 군집화  
# -----  
print("\n--- 평균 이동 알고리즘 수행 ---")  
  
# 1) 대역폭(Bandwidth) 추정  
# quantile: 데이터를 탐색할 범위 (0~1 사이). 클수록 군집 수가 적어지고, 작을수록 많아짐.  
# 보통 0.2(상위 20%) ~ 0.3 정도로 설정합니다. 군집이 너무 많으면 이 값을 올리세요.  
my_quantile = 0.05  
bandwidth = estimate_bandwidth(X_scaled, quantile=my_quantile)  
  
print(f"    -> 추정된 대역폭(Bandwidth): {bandwidth:.4f} (quantile={my_quantile})")  
  
# 2) 모델 학습  
ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)  
ms.fit(X_scaled)  
  
# 3) 라벨 및 중심점 추출  
labels = ms.labels_  
cluster_centers = ms.cluster_centers_  
n_clusters_ = len(np.unique(labels))  
  
df['Cluster_MeanShift'] = labels  
  
print(f"✅ 군집화 완료 발견된 군집 수: {n_clusters_}개")
```

```
--- 평균 이동 알고리즘 수행 ---  
    -> 추정된 대역폭(Bandwidth): 6.2732 (quantile=0.05)  
✅ 군집화 완료 발견된 군집 수: 6개
```

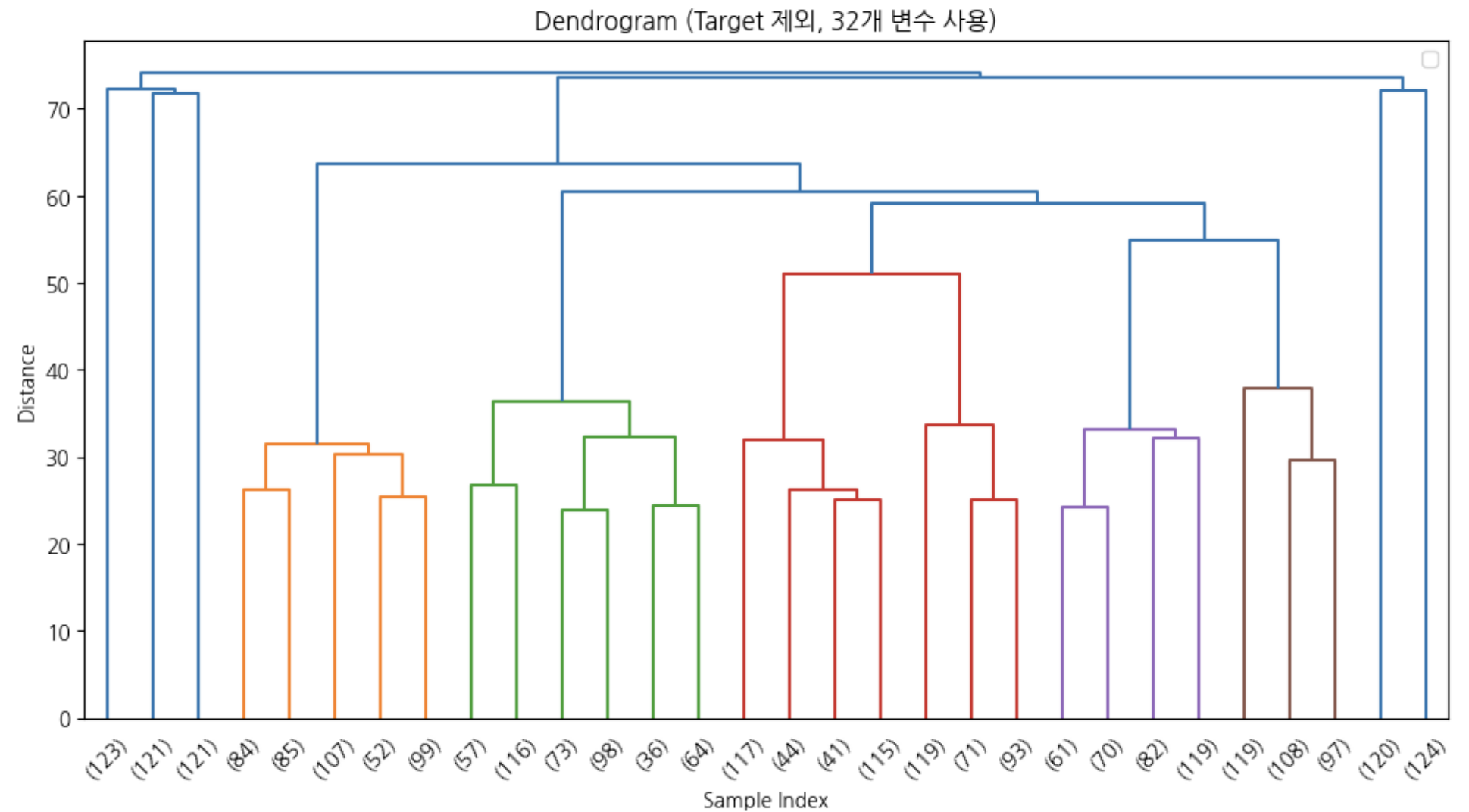
```
[군집별 데이터 개수]  
Cluster_MeanShift  
0      2511  
1       70  
2       66  
3       49  
4       34  
5        6  
Name: count, dtype: int64
```

대부분의 데이터가
한 군집에 존재!

분석 방법론

클러스터링 기법 - 계층적 군집화

```
# -----  
# 2. 변수 분리 (target_col 설정)  
# -----  
print("\n--- 2. 변수 분리 (X: 군집화용, y: 분석용) ---")  
  
# 1) 타겟 컬럼(나중에 분석할 변수 - 순수익) 지정  
target_col = 'Net_Profit_USD'  
  
# 2) 모든 수치형 변수 선택  
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()  
  
# 3) X와 y 분리  
if target_col in df.columns:  
    # y: 타겟 변수 (분석용)  
    y = df[target_col]  
  
    # X: 타겟 변수를 제외한 나머지 수치형 변수들 (군집화용)  
    if target_col in numeric_cols:  
        numeric_cols.remove(target_col)  
  
    X = df[numeric_cols].copy()  
    print(f"✅ '{target_col}'을(를) y(분석용)로 분리하고, X(군집화용)에서 제외했습니다.")  
else:  
    print(f" '{target_col}' 컬럼이 데이터에 없습니다. 전체 데이터를 사용합니다.")  
    X = df[numeric_cols].copy()  
    y = None  
  
print(f" 군집화에 사용될 변수 목록 ({len(X.columns)}개):")  
print(X.columns.tolist())  
  
# -----  
# 4. 링크지 계산 및 덴드로그램  
# -----  
print("\n--- 4. 링크지 계산 및 덴드로그램 시각화 ---")  
# Ward 연결법  
Z = hierarchy.linkage(X_scaled, method='ward')  
print("✅ 'Ward' 링크지 계산 완료.")  
  
plt.figure(figsize=(12, 6))  
plt.title(f"Dendrogram (Target 제외, {len(X.columns)}개 변수 사용)")  
plt.ylabel('Distance')  
plt.xlabel('Sample Index')  
  
# 덴드로그램 그리기 (상위 30개만 요약 표시)  
hierarchy.dendrogram(Z, truncate_mode='lastp', p=30, show_leaf_counts=True)  
  
plt.legend()  
  
plt.show()
```



분석 방법론

클러스터링 기법 - 계층적 군집화

```
# -----  
# 최적의 K 찾기 (K=4부터 확인)  
# -----  
print("\n--- 최적의 K 찾기 (실루엣 계수, K=4부터) ---")  
  
# K를 4부터 6까지 바꿔가며 점수 확인  
start_k = 4 # 시작할 K값  
limit_k = 6 # 확인할 최대 K값  
best_k = 0  
best_score = -1  
  
# 결과 저장을 위한 리스트  
k_list = []  
score_list = []  
  
for k in range(start_k, limit_k + 1):  
    # fcluster를 사용하여 k개의 군집으로 자르기  
    labels_temp = hierarchy.fcluster(Z, t=k, criterion='maxclust')  
  
    # 실루엣 점수 계산  
    score = silhouette_score(X_scaled, labels_temp)  
  
    k_list.append(k)  
    score_list.append(score)  
  
    print(f"K={k} 일 때 실루엣 점수: {score:.4f}")  
  
    # 최고 점수 갱신 확인  
    if score > best_score:  
        best_score = score  
        best_k = k  
  
print(f"\n✅ 실루엣 계수 기준 최적의 K (범위 {start_k}~{limit_k}): {best_k} (점수: {best_score:.4f})")  
  
# -----  
# 실루엣 점수 시각화  
# -----  
plt.figure(figsize=(10, 5))  
plt.plot(k_list, score_list, marker='o', linestyle='--')  
plt.title(f'Silhouette Score vs K (Starting from {start_k})')  
plt.xlabel('Number of Clusters (K)')  
plt.ylabel('Silhouette Score')  
plt.axvline(x=best_k, color='r', linestyle='--', label=f'Best K={best_k}')  
plt.legend()  
plt.grid(True)  
plt.show()
```

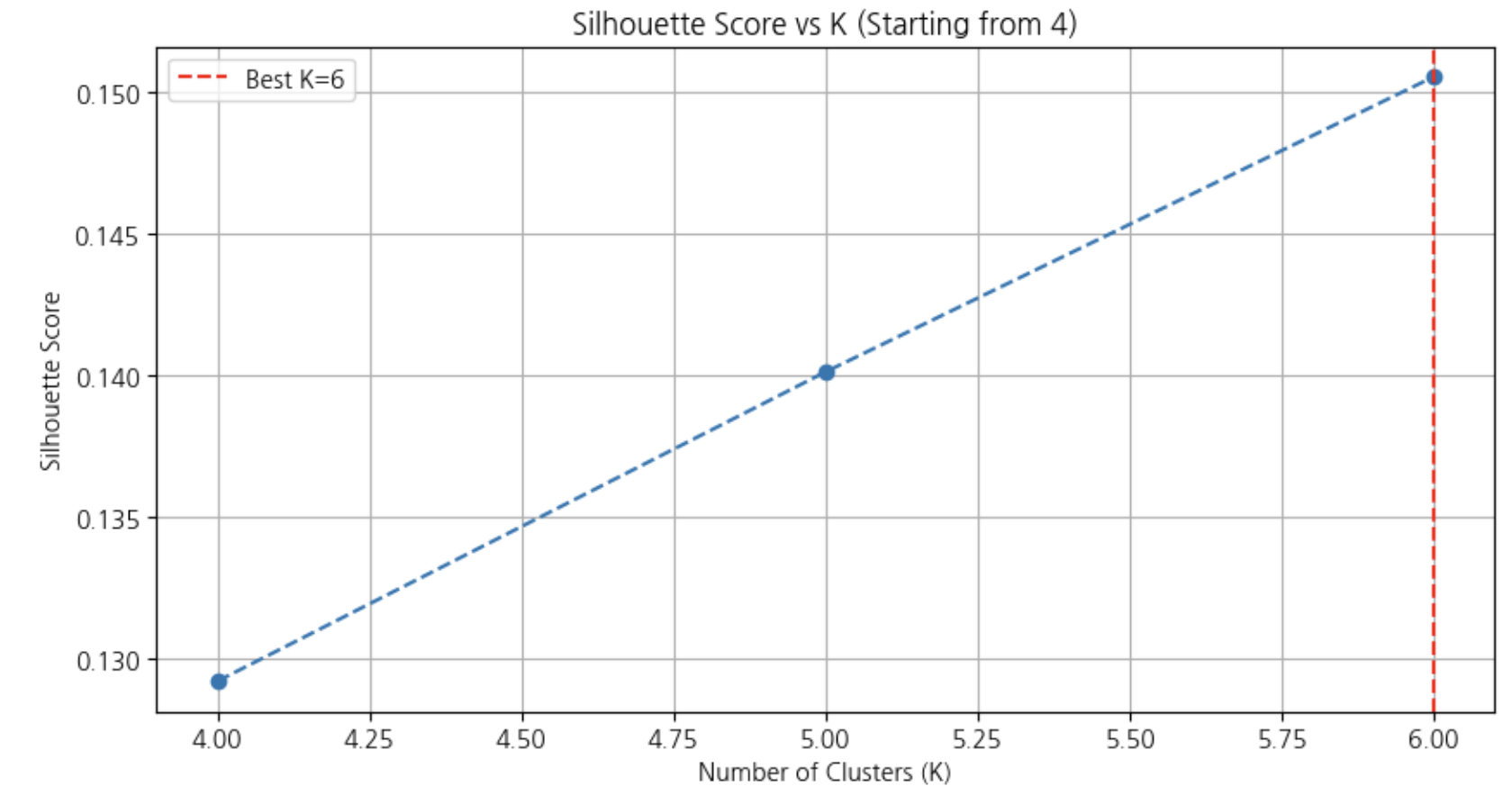
--- 최적의 K 찾기 (실루엣 계수, K=4부터) ---

K=4 일 때 실루엣 점수: 0.1292

K=5 일 때 실루엣 점수: 0.1401

K=6 일 때 실루엣 점수: 0.1505

✅ 실루엣 계수 기준 최적의 K (범위 4~6): 6 (점수: 0.1505)



실증 분석

군집분석 결과

- 계층적 군집화

```
# -----
# 5. 군집 할당 (k 설정)
# -----
# 덴드로그램을 보고 k값 결정
k = 6
print(f"\n--- k={k} 로 군집 할당 ---")

labels = hierarchy.fcluster(Z, t=k, criterion='maxclust')

# 원본 데이터프레임에 군집 라벨 저장
df['Cluster_Label'] = labels
print(f"✅ {k}개의 군집으로 나누어 'Cluster_Label' 컬럼에 저장했습니다.")

# -----
# 6. 결과 분석: 군집별 특성 및 수익 비교
# -----
print(f"\n--- 6. 군집별 특성 및 '{target_col}' 분석 ---")

if y is not None:
    print("[군집별 데이터 개수]")
    print(df['Cluster_Label'].value_counts().sort_index())

    print(f"\n[군집별 '{target_col}' (순수익) 평균 비교]")
    # 수익 평균을 기준으로 내림차순 정렬하여 확인
    profit_summary = df.groupby('Cluster_Label')[target_col].mean().sort_values(ascending=False)
    print(profit_summary)

    print(f"\n[군집별 '{target_col}' 상세 통계]")
    print(df.groupby('Cluster_Label')[target_col].describe())

    # 시각화 (Boxplot)
    plt.figure(figsize=(10, 6))
    sns.boxplot(x='Cluster_Label', y=target_col, data=df)
    plt.title(f"Cluster vs {target_col}")
    plt.show()
else:
    print("타겟 변수가 없어 분석을 생략합니다.")
```

```
--- 6. 군집별 특성 및 'Net_Profit_USD' 분석 ---
[군집별 데이터 개수]
Cluster_Label
1      123
2      121
3      121
4     2127
5      120
6      124
Name: count, dtype: int64

[군집별 'Net_Profit_USD' (순수익) 평균 비교]
Cluster_Label
1      313071.588234
6      288507.678439
4      266385.336792
3      266276.546110
2      244281.645493
5      214270.842709
Name: Net_Profit_USD, dtype: float64

[군집별 'Net_Profit_USD' 상세 통계]
```

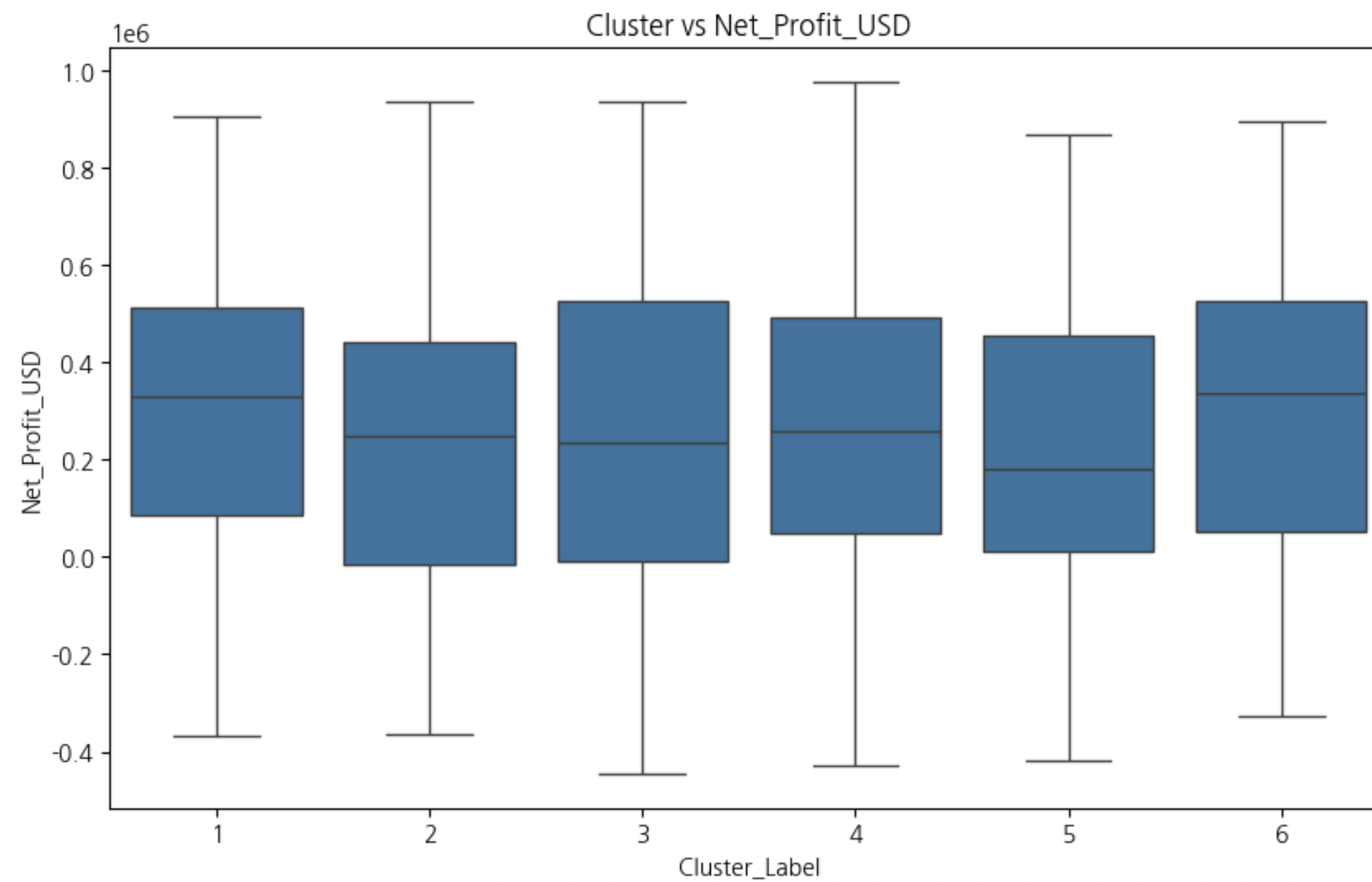
	count	mean	std	min	\
Cluster_Label					
1	123.0	313071.588234	291404.886919	-367850.1646	
2	121.0	244281.645493	313866.160808	-365486.4791	
3	121.0	266276.546110	342600.809270	-444584.0492	
4	2127.0	266385.336792	300340.093801	-428282.4624	
5	120.0	214270.842709	304052.909916	-419128.6395	
6	124.0	288507.678439	295239.358786	-326120.0016	

```

                25%                50%                75%                max
Cluster_Label
1      85944.499453    328892.908989    513486.138781    903772.017864
2     -15384.259090    249565.528049    441373.468277    934269.332572
3     -7726.622320    233547.765332    526539.662386    935957.241000
4     47542.889649    259070.744800    492047.396011    977168.394742
5     10779.506647    181567.351089    454935.452646    866827.163540
6     53351.431946    335124.071963    524693.734720    895828.021100
```


실증 분석

군집분석 결과 - 계층적 군집화



분석 결과

클러스터 간 비교

Cluster_Label	데이터 개수 (count)	Net_Profit_USD 평균 (USD)	비고
1	123	313,071.59	가장 높은 평균 수익
6	124	288,507.68	두 번째로 높은 평균 수익
4	2,127	266,385.34	가장 큰 군집 (대부분의 데이터)
3	121	266,276.55	평균 수익은 Cluster 4와 유사
2	121	244,281.65	네 번째로 낮은 평균 수익
5	120	214,270.84	가장 낮은 평균 수익

- Cluster 4는 전체의 약 77.7% 데이터를 포함하는 매우 큰 군집임
- 나머지 5개 군집(Cluster 1, 2, 3, 5, 6)은 각각 120개 내외의 데이터를 포함하는 작은 군집. 특정한 행동 패턴이나 속성에서 일반 군집(Cluster 4)과 뚜렷한 차이를 보이는 '특이한 그룹'일 가능성이 높음

실증 분석

클러스터 간 비교

변수 (범위)	Cluster 1 (고수익)	Cluster 5 (저수익)	Cluster 4 (일반)
Net_Profit_USD 평균 (USD)	313,072	214,271	266,385
Efficiency_nm_per_kWh (효율)	+0.0882	-0.0201	0.0000
Average_Load_Percentage (적재율)	+0.0993	-0.0601	-0.0151
Turnaround_Time_hours (회항 시간)	-0.0281	0.0018	0.0060
Speed_Over_Ground_knots (선속)	-0.0482	0.0014	-0.0146
Cargo_Weight_tons (화물 중량)	+0.0204	-0.0613	-0.0100
Ship_Type_nan (선종 정보 없음)	1.0000	0.0500	0.0000
Maintenance_Status_nan (정비 상태 없음)	0.0081	1.0000	0.0000
Weather_Condition_Calm (잔잔한 날씨)	0.3333	0.3917	0.3404
Route_Type_Long-haul (장거리 항로)	0.2114	0.3167	0.2661

- Cluster 4와 Cluster 1, 5를 비교 하여 주성분을 비교 가능
- 변수들은 스케일링되었으므로, 0에 가까우면 전체 평균 수준, 양수면 평균보다 높음, 음수면 평균보다 낮음을 의미

실증 분석

클러스터 간 비교

Cluster 1(최고 수익 그룹)

- 효율(Efficiency_nm_per_kWh) 평균이 +0.0882로, 6개 군집 중 가장 높은 에너지 효율성을 보였음. 이는 동일 연료로 더 먼 거리를 운항하여 운영 비용을 절감했음을 의미
- 높은 활용률: Average_Load_Percentage 평균이 +0.0993으로 가장 높았으며, 선속(Speed_Over_Ground_knots)은 평균보다 낮았음에도 불구하고 높은 적재율을 유지하여 운항당 수익을 극대화

Cluster 5(최저 수익 그룹)

- 모든 선박이 Maintenance_Status_nan(정비 상태 정보 결측) 값을 가짐. 이는 정비 기록이 누락된 선박들의 집합이며, 이러한 정보의 불완전성 자체가 낮은 수익과 연관될 수 있음을 시사
- 낮은 활용률: Average_Load_Percentage 평균이 -0.0601로 평균보다 낮아, 선박이 충분히 채워지지 않은 상태로 운항했음을 보여줌
- 비효율성: Efficiency_nm_per_kWh는 -0.0201로 평균보다 약간 낮았으며, Cargo_Weight_tons 역시 평균보다 낮아 운송 가치 및 효율이 전반적으로 저조

결론

PCA

```
# 파일 업로드
uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])
print("데이터 로드 완료:", df.shape)

# 군집 컬럼 자동 탐색
cluster_col = None
for c in df.columns:
    if "cluster" in c.lower():
        cluster_col = c
        break

print("군집 컬럼:", cluster_col)

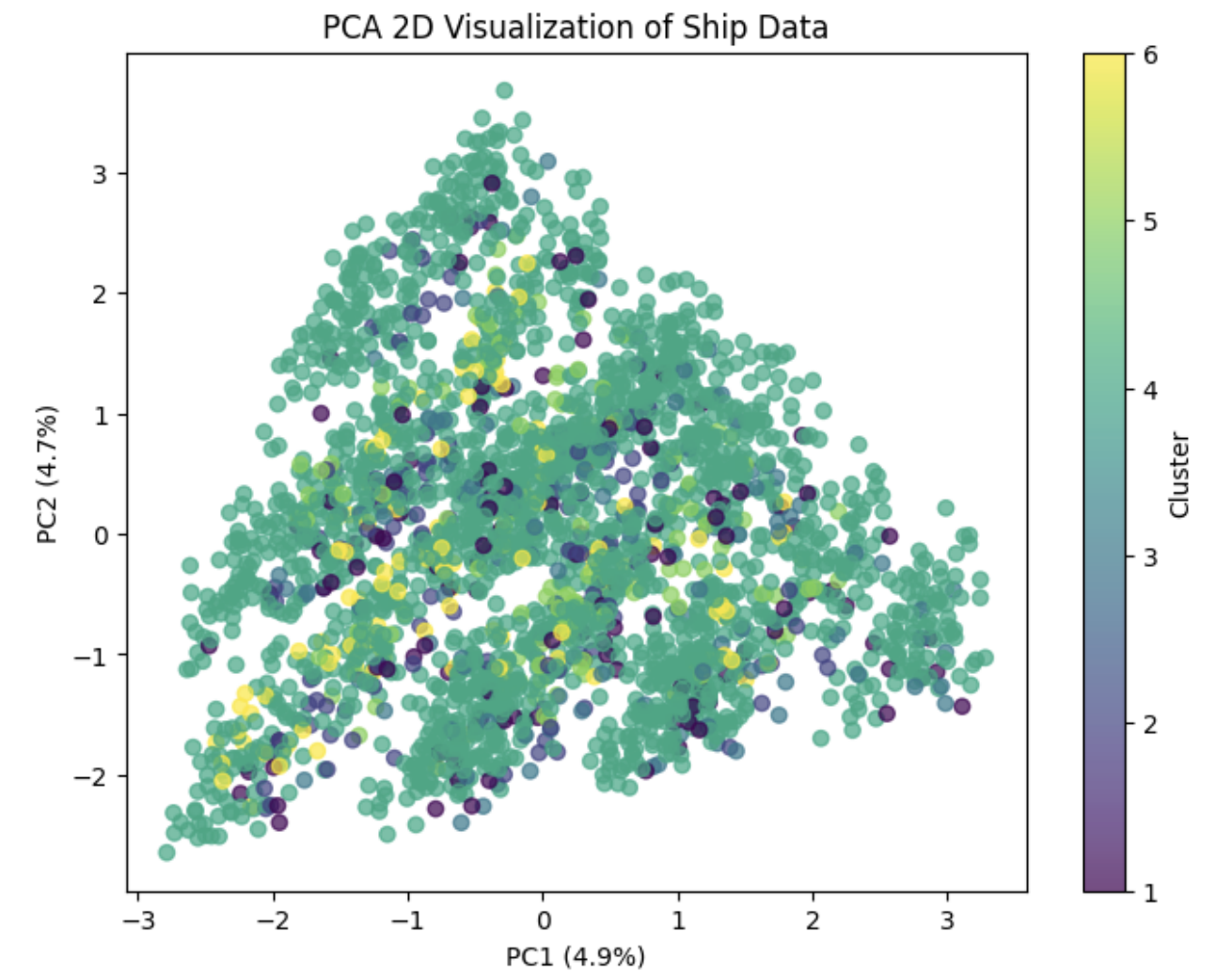
# PCA 대상 변수 (군집 제외)
X = df.drop(columns=[cluster_col])

# 표준화
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# PCA 2D
pca = PCA(n_components=2)
pca_result = pca.fit_transform(X_scaled)

# 시각화
plt.figure(figsize=(8,6))
plt.scatter(pca_result[:,0], pca_result[:,1], c=df[cluster_col], cmap='viridis', alpha=0.7)
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)")
plt.title("PCA 2D Visualization of Ship Data")
plt.colorbar(label="Cluster")
plt.show()
```

데이터 로드 완료: (2736, 34)
군집 컬럼: Cluster_Label



결론

PCA

```
# =====
# 📌 PCA 변수 기여도 분석 (Loadings)
# =====
from google.colab import files
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 파일 업로드
uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])

# 군집 컬럼 자동 탐색
cluster_col = None
for c in df.columns:
    if "cluster" in c.lower():
        cluster_col = c
        break

X = df.drop(columns=[cluster_col])

# 표준화
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# PCA 2개 성분 계산
pca = PCA(n_components=2)
pca.fit(X_scaled)

# 변수 기여도 출력
loadings = pd.DataFrame(
    pca.components_.T,
    columns=["PC1", "PC2"],
    index=X.columns
)

print("📌 PCA Loading 값 (변수 기여도):")
print(loadings.sort_values("PC1", ascending=False).head(10))
print("\n[PC2 기여도 TOP 10]")
print(loadings.sort_values("PC2", ascending=False).head(10))
```

📌 PCA Loading 값 (변수 기여도):

	PC1	PC2
Weather_Condition_Calm	0.469960	-0.207183
Maintenance_Status_Critical	0.465520	-0.147414
Engine_Type_Heavy Fuel Oil (HF0)	0.338193	-0.047583
Ship_Type_Container Ship	0.089340	-0.032869
Efficiency_nm_per_kWh	0.063928	-0.088459
Ship_Type_Bulk Carrier	0.058104	0.167720
Route_Type_nan	0.055005	-0.038294
Engine_Type_Diesel	0.053361	0.262362
Route_Type_Short-haul	0.053105	0.156253
Route_Type_Long-haul	0.036241	0.004702

[PC2 기여도 TOP 10]

	PC1	PC2
Maintenance_Status_Fair	-0.080235	0.518200
Weather_Condition_Moderate	-0.217513	0.483852
Engine_Type_Diesel	0.053361	0.262362
Ship_Type_Bulk Carrier	0.058104	0.167720
Route_Type_Short-haul	0.053105	0.156253
Ship_Type_Tanker	-0.047052	0.032682
Seasonal_Impact_Score	-0.015390	0.030900
Speed_Over_Ground_knots	0.031467	0.022264
Distance_Traveled_nm	-0.039287	0.019135
Draft_meters	-0.036290	0.015995

결론

PCA

PC1 (첫 번째 주성분)

- 'Weather_Condition_Calm' (0.469960), 'Maintenance_Status_Critical' (0.465520), 'Engine_Type_Heavy Fuel Oil (HFO)' (0.338193) 등이 양의 값으로 높은 기여도를 보임. 이는 첫 번째 주성분이 주로 '고요한 날씨', '심각한 유지보수 상태', '중유 엔진 사용'과 같은 특성들과 강하게 연관되어 있음을 시사함
- PC1은 전반적으로 선박의 '운항 환경', '유지보수 상태', '엔진 유형'과 같은 요인들의 주요 변화 방향을 나타낼 가능성이 높음

PC2 (두 번째 주성분)

- 'Maintenance_Status_Fair' (0.518200), 'Weather_Condition_Moderate' (0.483852), 'Engine_Type_Diesel' (0.262362) 등이 양의 값으로 높은 기여도를 보임. 이는 두 번째 주성분이 주로 '양호한 유지보수 상태', '적당한 날씨', '디젤 엔진 사용'과 같은 특성들과 강하게 연관되어 있음을 나타냄
- PC2는 PC1과는 다른 차원의 주요 변화, 예를 들어 '일반적인 운항 조건'이나 '특정 엔진 유형'의 영향을 반영할 수 있음.

**PC1은 부정적이거나 특정 상황과 관련된 요인들을 묶는 경향이 있는 반면,
PC2는 비교적 긍정적이거나 다른 유형의 운항 조건을 설명하는 요인들을 나타낸다.**

결론

1. 정비 관리의 중요성 (Maintenance)

- 최저 수익 그룹(C5)은 정비 정보가 누락(Maintenance_Status_nan)된 선박들의 집합으로, 이는 정비의 불확실성이 낮은 수익과 연관됨을 강하게 시사함
- PC2에서 'Maintenance_Status_Fair'가 긍정적인 운항 조건(PC2)에 기여하는 것과 대조를 이루며, 적절하고 기록된 정비 상태가 수익성 확보의 기본 전제임을 검증

2. 활용률 및 부하율 최적화 (Load Percentage)

- 최고 수익 그룹(C1)은 평균보다 높은 부하율을 유지하여 적재율당 운항 수익을 극대화, 최저 수익 그룹(C5)은 선박이 채워지지 않은 상태로 운항하여 비용 효율성이 저하
- 선박의 수익성을 높이는 데 있어 단순히 운항하는 것을 넘어, 최대 적재량에 가깝게 채워서 운항하는 것이 핵심

3. 에너지 효율 관리 (Efficiency)

- C1은 높은 에너지 효율을, C5는 낮은 에너지 효율을 보임
- 효율성은 엔진 유형(HFO/Diesel, PC1/PC2) 및 운항 환경(Weather, PC1/PC2) 등 다양한 요인의 복합적인 결과임
- 고효율 운항은 동일 연료로 더 많은 거리를 운항하는 직접적인 비용 절감 효과를 가져오며 수익성을 높임

Thanks
